Resource-Constrained Project Scheduling Problem (RCPSP) — Metaheuristic Implementations

This document consolidates three distinct Python implementations for solving the Resource-Constrained Project Scheduling Problem (RCPSP):

1. **Implementation 1 (ga_rcpsp.py)**: Genetic Algorithm using an interval-overlap decoder and topological precedence-repair mutation.
2. **Implementation 2 (rcpsp_numpy_ga.py)**: Genetic Algorithm using a NumPy-based timeline Serial Generation Scheme (SGS) and dummy-bounded chromosome crossover.
3. **Implementation 3 (rcpsp_simulated_annealing.py)**: Simulated Annealing using an object-oriented multi-resource SGS and adjacent precedence-feasible swap neighborhood moves.

1. Comparative Architecture Overview

Feature	Implementation 1 (ga_rcpsp.py)	Implementation 2 (rcpsp_numpy_ga.py)	Implementation 3 (rcpsp_simulated_annealing.py)
Metaheuristic	Genetic Algorithm (GA)	Genetic Algorithm (GA)	Simulated Annealing (SA)
Problem Representation	Activity priority permutation (topological repair on decode)	Activity list with fixed dummy boundaries $[0, \ldots, n -$	Precedence-feasible activity permutation list
Schedule Generation Scheme (SGS)	Dynamic interval conflict checking (used intervals list)	Discrete time-indexed vector via NumPy array (resource_timeline)	Multi-resource discrete timeline array with lazy extension
Precedence Handling	Graph validation + topological repair	Pre-validation check / reject invalid candidate	Adjacent-swap feasibility check against direct successors

Crossover / Neighborhood	Order Crossover (OX) across all tasks	OX on interior slice [1:-1], pinning dummy tasks	Adjacent-feasible swap ($i, i+$)
Selection / Acceptance	Tournament Selection ($k =$) + Elitism	Fitness rank sorting + Top-20% Elitism + Tournament	Metropolis Acceptance Criterion ($e^{-\Delta/T}$)
Multi-Resource Support	Single renewable resource	Single renewable resource	Arbitrary number of renewable resources ($R_1, R_2, \ldots, R_m$)

2. Implementation 1: Genetic Algorithm with Precedence Repair (ga_rcpsp.py)

Characteristics

- Explicit 8-task dependency graph.
- Interval-overlap checking for resource constraint satisfaction.
- Swap mutation followed by a deterministic topological repair mechanism.

"""
GA-based Resource-Constrained Project Scheduling Problem (RCPSP)
Project: AI-Enhanced Metaheuristics Control for Project Scheduling

Run:
  python ga_rcpsp.py
"""

import random

# ----------------------------
# Example RCPSP project data
# ----------------------------
# task: (duration, resource_demand, predecessors)
TASKS = {
  1: (4, 2, []),
  2: (3, 2, [1]),
  3: (5, 3, [1]),

```
    4: (2, 2, [2]),
    5: (4, 2, [2]),
    6: (3, 2, [3]),
    7: (2, 3, [4, 6]),
    8: (1, 1, [5, 7]),
}

RESOURCE_CAPACITY = 5

POPULATION_SIZE = 40
GENERATIONS = 100
CROSSOVER_RATE = 0.85
MUTATION_RATE = 0.15
ELITE_SIZE = 2
TOURNAMENT_SIZE = 3
RANDOM_SEED = 42


def random_topological_order(tasks):
    """Create a valid chromosome that respects precedence ordering."""
    remaining = set(tasks)
    order = []

    while remaining:
        available = [
            t for t in remaining
            if all(p in order for p in tasks[t][2])
        ]
        if not available:
            raise ValueError("Invalid dependency graph: cycle detected.")
        random.shuffle(available)
        order.append(available[0])
        remaining.remove(available[0])

    return order


def decode(chromosome):
    """
    Convert a priority permutation into a feasible schedule.
    Each task starts at the earliest time satisfying:
      1. all predecessors are complete
      2. resource capacity is not exceeded
```

```
    """
    schedule = {}
    used = []

    # Process tasks in precedence-feasible priority order.
    # This also makes the decoder robust if a crossover creates
    # a permutation that is not itself topologically sorted.
    remaining = list(chromosome)

    while remaining:
        available = [
            task for task in remaining
            if all(p in schedule for p in TASKS[task][2])
        ]

        if not available:
            raise ValueError("Invalid dependency graph: cycle detected.")

        # Preserve chromosome priority among currently available tasks.
        task = min(available, key=remaining.index)
        remaining.remove(task)

        duration, demand, predecessors = TASKS[task]

        earliest = 0
        if predecessors:
            earliest = max(schedule[p]["finish"] for p in predecessors)

        start = earliest

        while True:
            finish = start + duration
            conflict = False

            for s, f, d in used:
                # overlap exists if time intervals intersect
                if start < f and finish > s:
                    if d + demand > RESOURCE_CAPACITY:
                        conflict = True
                        break

            if not conflict:
                break
```

```
            start += 1

        schedule[task] = {
            "start": start,
            "finish": finish,
            "duration": duration,
            "resource": demand,
        }
        used.append((start, finish, demand))

    makespan = max(v["finish"] for v in schedule.values())
    return schedule, makespan


def fitness(chromosome):
    """Lower makespan is better; convert it to a maximization score."""
    _, makespan = decode(chromosome)
    return 1.0 / (1.0 + makespan)


def tournament_selection(population):
    candidates = random.sample(population, TOURNAMENT_SIZE)
    return min(candidates, key=lambda c: decode(c)[1])


def order_crossover(parent1, parent2):
    """Order crossover (OX) for permutation chromosomes."""
    n = len(parent1)
    a, b = sorted(random.sample(range(n), 2))

    child = [None] * n
    child[a:b + 1] = parent1[a:b + 1]

    remaining = [x for x in parent2 if x not in child]
    pos = [i for i in range(n) if child[i] is None]

    for i, value in zip(pos, remaining):
        child[i] = value

    return child


def precedence_safe_mutation(chromosome):
```

```
    """Swap mutation followed by a topological repair."""
    child = chromosome[:]
    i, j = random.sample(range(len(child)), 2)
    child[i], child[j] = child[j], child[i]

    # Repair into a valid topological order while preserving the
    # mutated chromosome's priority as much as possible.
    priority = {task: idx for idx, task in enumerate(child)}
    remaining = set(child)
    repaired = []

    while remaining:
        available = [
            t for t in remaining
            if all(p in repaired for p in TASKS[t][2])
        ]
        available.sort(key=lambda t: priority[t])
        repaired.append(available[0])
        remaining.remove(available[0])

    return repaired


def genetic_algorithm():
    random.seed(RANDOM_SEED)

    population = [
        random_topological_order(TASKS)
        for _ in range(POPULATION_SIZE)
    ]

    best = min(population, key=lambda c: decode(c)[1])

    print("=== GENETIC ALGORITHM FOR RCPSP ===")
    print(f"Tasks: {len(TASKS)}")
    print(f"Resource capacity: {RESOURCE_CAPACITY}")
    print(f"Population: {POPULATION_SIZE}")
    print(f"Generations: {GENERATIONS}")
    print()

    for generation in range(1, GENERATIONS + 1):
        population.sort(key=lambda c: decode(c)[1])
```

```
    if decode(population[0])[1] < decode(best)[1]:
        best = population[0][:]

    if generation == 1 or generation % 10 == 0:
        print(
            f"Generation {generation:3d} | "
            f"Best makespan = {decode(best)[1]}"
        )

    new_population = [c[:] for c in population[:ELITE_SIZE]]

    while len(new_population) < POPULATION_SIZE:
        p1 = tournament_selection(population)
        p2 = tournament_selection(population)

        if random.random() < CROSSOVER_RATE:
            child = order_crossover(p1, p2)
        else:
            child = p1[:]

        if random.random() < MUTATION_RATE:
            child = precedence_safe_mutation(child)

        new_population.append(child)

    population = new_population

schedule, makespan = decode(best)

print("\n=== FINAL RESULT ===")
print("Best chromosome:", best)
print("Minimum makespan:", makespan)

print("\nTask schedule:")
print("Task | Start | Finish | Duration | Resource")
print("-----------------------------------------------")
for task in sorted(schedule):
    s = schedule[task]
    print(
        f"{task:4d} | {s['start']:5d} | {s['finish']:6d} | "
        f"{s['duration']:8d} | {s['resource']:8d}"
    )
```

```
    return best, schedule, makespan


if __name__ == "__main__":
    genetic_algorithm()
```

3. Implementation 2: NumPy Timeline Serial Generation Scheme GA (rcpsp_numpy_ga.py)

Characteristics

- Uses dummy source (0) and sink (4) tasks.
- Tracks resource availability via a 1D NumPy timeline array.
- OX crossover isolates interior non-dummy activities.
- Swap mutation validates feasibility post-swap and discards invalid alterations.

```
"""
RCPSP Genetic Algorithm with NumPy Timeline Serial Generation Scheme (SGS)
Transcribed from source PDF.
"""

import random
import numpy as np

# ================================================
# 1. SAMPLE PROBLEM DEFINITION (RCPSP)
# ================================================
# 5 Activities (0 = Start Dummy, 4 = End Dummy)
# Precedence map: key must be completed before values can start
PRECEDENCE = {
    0: [1, 2],
    1: [3],
    2: [3],
    3: [4],
    4: []
}

DURATIONS = {0: 0, 1: 4, 2: 2, 3: 3, 4: 0}
RESOURCE_REQS = {0: 0, 1: 4, 2: 3, 3: 5, 4: 0}
TOTAL_RESOURCES_AVAILABLE = 6
```

```
# ================================================
# 2. SERIAL GENERATION SCHEME (SGS)
# ================================================
def serial_generation_scheme(chromosome):
    """
    Decodes an activity-list chromosome into a valid schedule
    calculating the total project duration (makespan).
    """
    start_times = {}
    end_times = {}

    # Track resource consumption across a timeline maxed out at theoretical horizon
    max_horizon = sum(DURATIONS.values()) + 1
    resource_timeline = np.zeros(max_horizon)

    for activity in chromosome:
        # Step A: Earliest time based strictly on Precedence Constraints
        earliest_start = 0
        for pred, successors in PRECEDENCE.items():
            if activity in successors:
                earliest_start = max(earliest_start, end_times.get(pred, 0))

        # Step B: Earliest time slot satisfying Resource Constraints
        current_start = earliest_start
        duration = DURATIONS[activity]
        req = RESOURCE_REQS[activity]

        if duration == 0:  # Dummy start/end activities
            start_times[activity] = current_start
            end_times[activity] = current_start
            continue

        while True:
            # Check resource availability in the window [current_start, current_start + duration]
            window = resource_timeline[current_start:current_start + duration]
            if np.all(window + req <= TOTAL_RESOURCES_AVAILABLE):
                # Allocate resources
                resource_timeline[current_start:current_start + duration] += req
                start_times[activity] = current_start
                end_times[activity] = current_start + duration
                break
            current_start += 1
```

```
    return end_times[chromosome[-1]]


# ================================================
# 3. GENETIC ALGORITHM OPERATORS
# ================================================
def generate_valid_chromosome():
    """Generates an activity list respecting topological precedence."""
    activities = list(DURATIONS.keys())
    chromosome = [0]  # Always starts with project origin
    eligible = [act for act in activities if act != 0]

    while eligible:
        valid_choices = []
        for act in eligible:
            is_valid = True
            for pred, successors in PRECEDENCE.items():
                if act in successors and pred not in chromosome:
                    is_valid = False
                    break
            if is_valid:
                valid_choices.append(act)

        chosen = random.choice(valid_choices)
        chromosome.append(chosen)
        eligible.remove(chosen)

    return chromosome


def crossover(parent1, parent2):
    """Order-Based Crossover (OX) adapted to preserve dummy boundaries."""
    # Fix the start and end dummies
    core1 = parent1[1:-1]
    core2 = parent2[1:-1]
    size = len(core1)

    idx1, idx2 = sorted(random.sample(range(size), 2))

    child1_core = [None] * size
    # Inherit slice from parent 1
    child1_core[idx1:idx2] = core1[idx1:idx2]
```

```
    # Fill remaining from parent 2 tracking relative order
    p2_pointer = 0
    for i in range(size):
        if child1_core[i] is None:
            while core2[p2_pointer] in child1_core:
                p2_pointer += 1
            child1_core[i] = core2[p2_pointer]

    return [0] + child1_core + [len(DURATIONS) - 1]


def mutate(chromosome, mutation_rate=0.1):
    """Swap mutation that verifies precedence validity post-swap."""
    if random.random() > mutation_rate:
        return chromosome

    mutated = list(chromosome)
    # Don't touch index 0 (Start) and last index (End)
    idx1, idx2 = random.sample(range(1, len(chromosome) - 1), 2)

    # Swap elements
    mutated[idx1], mutated[idx2] = mutated[idx2], mutated[idx1]

    # Verify if swap broke precedence rules
    for i, act in enumerate(mutated):
        for pred, successors in PRECEDENCE.items():
            if act in successors:
                pred_idx = mutated.index(pred)
                if pred_idx > i:
                    return chromosome  # Invalid: Return untouched

    return mutated


# ==========================================
# 4. MAIN OPTIMIZATION LOOP
# ==========================================
def genetic_algorithm(pop_size=30, generations=20, mutation_rate=0.15):
    population = [generate_valid_chromosome() for _ in range(pop_size)]
    print(f"Starting Genetic Algorithm optimization across {generations} generations...\n")

    for gen in range(generations):
```

```
        fitness_scores = [serial_generation_scheme(chrom) for chrom in population]

        sorted_indices = np.argsort(fitness_scores)
        population = [population[i] for i in sorted_indices]
        fitness_scores = [fitness_scores[i] for i in sorted_indices]

        if (gen + 1) % 5 == 0 or gen == 0:
            print(f"Generation {gen+1:02d} | Best Makespan Found: {fitness_scores[0]} days")

        # Elitism: retain top 20%
        elite_count = int(pop_size * 0.2)
        next_generation = population[:elite_count]

        while len(next_generation) < pop_size:
            p1, p2 = random.sample(population[:int(pop_size * 0.6)], 2)
            child = crossover(p1, p2)
            child = mutate(child, mutation_rate)
            next_generation.append(child)

        population = next_generation

    best_chromosome = population[0]
    best_makespan = serial_generation_scheme(best_chromosome)

    print("\n" + "=" * 40)
    print("OPTIMIZATION COMPLETE")
    print(f"Optimal Scheduling Order: {best_chromosome}")
    print(f"Minimized Project Makespan: {best_makespan} days")
    print("=" * 40)

    return best_chromosome, best_makespan


if __name__ == "__main__":
    genetic_algorithm(pop_size=30, generations=20, mutation_rate=0.15)
```

4. Implementation 3: Multi-Resource Simulated Annealing (rcpsp_simulated_annealing.py)

Characteristics

- Encapsulated RCPSPInstance class supporting arbitrary renewable resource types ($R_1, R_2, \ldots, R_m$).
- Adjacent precedence-feasible swap neighborhood generator (swap_neighbor).
- Geometric temperature cooling ($T_{k+1} = \alpha \cdot T_k$) with Metropolis acceptance criterion and convergence counters.

"""
Simulated Annealing for the Resource-Constrained Project Scheduling Problem (RCPSP)

Consistent with the base paper's (Mitsou & Koulinas) SA baseline, extended here to
match the project's structure (activity list representation + serial SGS decode).

Encoding:
  A candidate solution is an "activity list" (a permutation of activity IDs that
  respects precedence, i.e. a valid topological order). This is decoded into a
  schedule using the Serial Schedule Generation Scheme (SGS), which respects both
  precedence and resource constraints.

Neighborhood move:
  Swap two adjacent-feasible activities in the activity list (a swap is only
  accepted as a candidate if it doesn't break precedence feasibility).

Objective:
  Minimize project makespan (completion time of the last activity).
"""

```
import math
import random
from copy import deepcopy


# ---------------------------------------------------------------------
# Problem definition helpers
# ---------------------------------------------------------------------

class RCPSPInstance:
    """
    activities: list of activity ids, e.g. [0, 1, 2, ..., n-1]
                0 is typically a dummy start, n-1 a dummy end (duration 0).
    durations:  dict {activity_id: duration}
    successors: dict {activity_id: [list of successor ids]}   (precedence)
    demands:    dict {activity_id: [resource_1_demand, resource_2_demand, ...]}
```

```
    capacities: list of resource capacities, e.g. [R1_cap, R2_cap, ...]
    """
    def __init__(self, activities, durations, successors, demands, capacities):
        self.activities = activities
        self.durations = durations
        self.successors = successors
        self.demands = demands
        self.capacities = capacities
        self.predecessors = {a: [] for a in activities}
        for a, succs in successors.items():
            for s in succs:
                self.predecessors[s].append(a)


# ---------------------------------------------------------------------------
# Serial Schedule Generation Scheme (decodes activity list -> schedule)
# ---------------------------------------------------------------------------

def serial_sgs(instance: RCPSPInstance, activity_list):
    """
    Decode a precedence-feasible activity list into start/finish times using
    the serial SGS, respecting renewable resource capacities.
    Returns (start_times, finish_times, makespan).
    """
    start = {}
    finish = {}
    # resource_usage[t] = list of resource usage at time t (extended lazily)
    resource_usage = []

    def usage_at(t, r_idx):
        if t < len(resource_usage):
            return resource_usage[t][r_idx]
        return 0

    def add_usage(t, demands):
        while len(resource_usage) <= t:
            resource_usage.append([0] * len(instance.capacities))
        for r_idx, d in enumerate(demands):
            resource_usage[t][r_idx] += d

    for act in activity_list:
        dur = instance.durations[act]
        preds = instance.predecessors[act]
```

```
        earliest = max([finish[p] for p in preds], default=0)

        demands = instance.demands.get(act, [0] * len(instance.capacities))

        # find first feasible start time >= earliest respecting resource caps
        t = earliest
        while True:
            feasible = True
            for tau in range(t, t + dur):
                for r_idx, cap in enumerate(instance.capacities):
                    if usage_at(tau, r_idx) + demands[r_idx] > cap:
                        feasible = False
                        break
                if not feasible:
                    break
            if feasible:
                break
            t += 1

        start[act] = t
        finish[act] = t + dur
        if dur > 0:
            for tau in range(t, t + dur):
                add_usage(tau, demands)

    makespan = max(finish.values())
    return start, finish, makespan


# ---------------------------------------------------------------------------
# Activity list utilities
# ---------------------------------------------------------------------------

def random_topological_order(instance: RCPSPInstance):
    """Generate a random precedence-feasible activity list."""
    remaining = set(instance.activities)
    indegree = {a: len(instance.predecessors[a]) for a in instance.activities}
    ready = [a for a in remaining if indegree[a] == 0]
    order = []
    while ready:
        random.shuffle(ready)
        a = ready.pop()
        order.append(a)
```

```
        remaining.remove(a)
        for s in instance.successors.get(a, []):
            indegree[s] -= 1
            if indegree[s] == 0:
                ready.append(s)
    return order


def swap_neighbor(instance: RCPSPInstance, activity_list):
    """
    Produce a neighbor by swapping two positions i, i+1 in the list,
    only if doing so keeps the list precedence-feasible.
    """
    new_list = activity_list[:]
    n = len(new_list)
    attempts = 0
    while attempts < n * 2:
        i = random.randint(0, n - 2)
        a, b = new_list[i], new_list[i + 1]
        # swap is feasible only if 'a' is not a required predecessor of 'b'
        if b not in instance.successors.get(a, []):
            new_list[i], new_list[i + 1] = b, a
            return new_list
        attempts += 1
    return new_list  # fallback: no swap found, return unchanged copy


# ---------------------------------------------------------------------------
# Simulated Annealing core
# ---------------------------------------------------------------------------

def simulated_annealing(
    instance: RCPSPInstance,
    initial_temp=1000.0,
    final_temp=1.0,
    cooling_rate=0.95,
    iterations_per_temp=50,
    max_no_improve=200,
    seed=None,
    verbose=False,
):
    """
    Standard SA loop:
```

```
  - geometric cooling: T = T * cooling_rate
  - Metropolis acceptance criterion for worse solutions
  - early stop if no improvement for `max_no_improve` outer iterations

Returns dict with best_solution (activity list), best_makespan,
and history (list of (temperature, best_makespan_so_far)) for plotting.
"""
if seed is not None:
    random.seed(seed)

current = random_topological_order(instance)
_, _, current_cost = serial_sgs(instance, current)

best = current[:]
best_cost = current_cost

temp = initial_temp
history = []
no_improve_counter = 0

while temp > final_temp and no_improve_counter < max_no_improve:
    improved_this_temp = False

    for _ in range(iterations_per_temp):
        candidate = swap_neighbor(instance, current)
        _, _, candidate_cost = serial_sgs(instance, candidate)

        delta = candidate_cost - current_cost

        if delta < 0:
            # improvement: always accept
            current, current_cost = candidate, candidate_cost
            if current_cost < best_cost:
                best, best_cost = current[:], current_cost
                improved_this_temp = True
        else:
            # worse solution: accept with Metropolis probability
            if delta == 0:
                accept_prob = 0.5
            else:
                accept_prob = math.exp(-delta / max(temp, 1e-9))
            if random.random() < accept_prob:
                current, current_cost = candidate, candidate_cost
```

```
        history.append((temp, best_cost))
        if verbose:
            print(f"T={temp:8.2f}  best_makespan={best_cost}")

        temp *= cooling_rate
        no_improve_counter = 0 if improved_this_temp else no_improve_counter + 1

    return {
        "best_activity_list": best,
        "best_makespan": best_cost,
        "history": history,
    }


# ---------------------------------------------------------------------------
# Example usage
# ---------------------------------------------------------------------------

if __name__ == "__main__":
    # Toy 6-activity instance: 0 = start, 5 = end (dummy, duration 0)
    activities = [0, 1, 2, 3, 4, 5]
    durations = {0: 0, 1: 4, 2: 2, 3: 3, 4: 5, 5: 0}
    successors = {
        0: [1, 2],
        1: [3],
        2: [3, 4],
        3: [5],
        4: [5],
        5: [],
    }
    demands = {
        0: [0], 1: [2], 2: [1], 3: [2], 4: [1], 5: [0],
    }
    capacities = [3]  # single renewable resource, capacity 3

    instance = RCPSPInstance(activities, durations, successors, demands, capacities)

    result = simulated_annealing(
        instance,
        initial_temp=500,
        final_temp=1,
        cooling_rate=0.9,
```

```
    iterations_per_temp=30,
    max_no_improve=15,
    seed=42,
    verbose=True,
)

print("\nBest activity list:", result["best_activity_list"])
print("Best makespan:", result["best_makespan"])
```